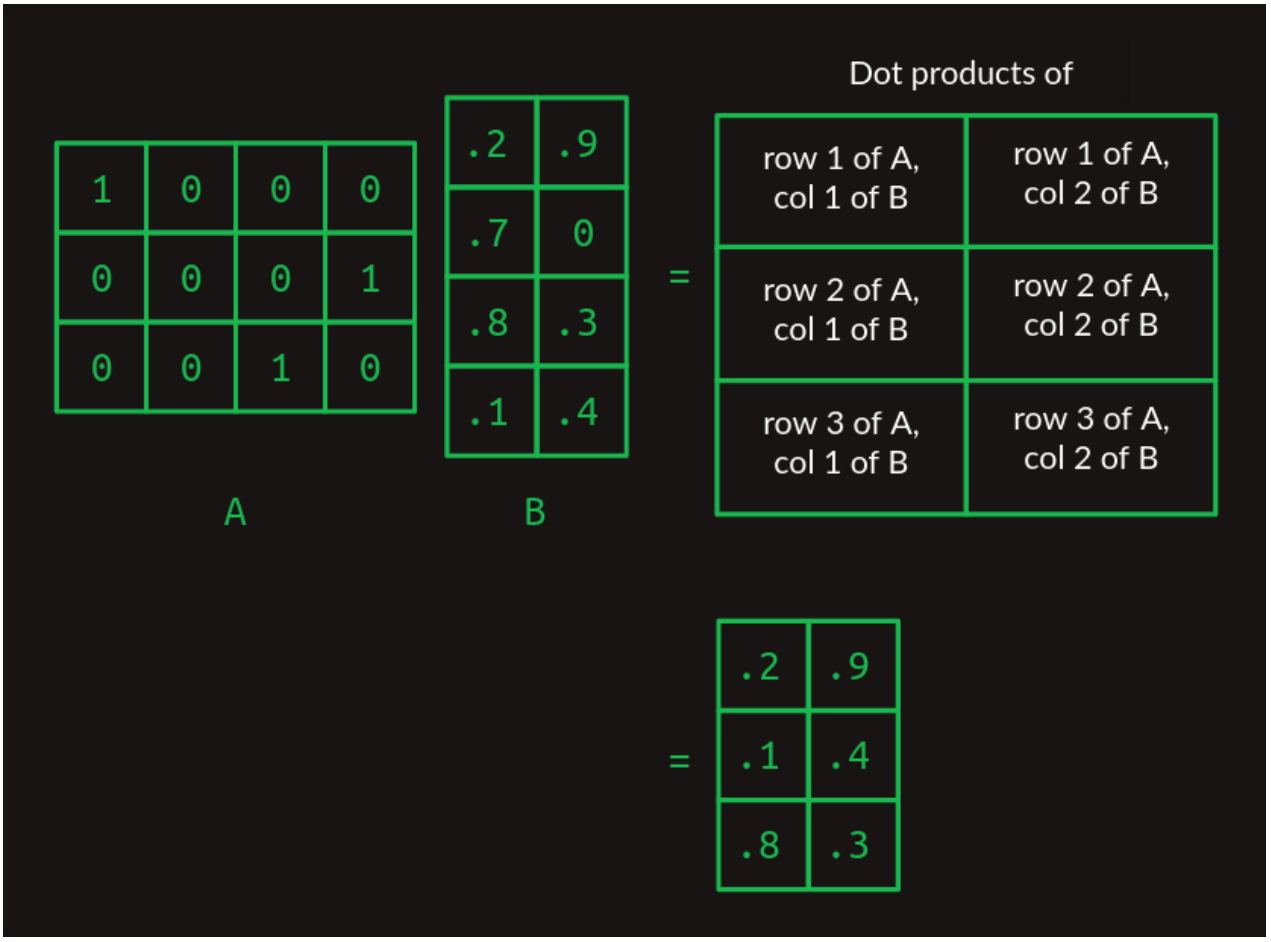
Note of *Transformers from Scratch*

- **One-hot encoding, Dot product, and Matrix multiplication**
  - *(This will provide a one-hot encoding example of a section above, which is more intuitive.)*
  - Similar to the idea of "**Math basics: dot product can reflect 'similarities'**" section of **Intuitive Understanding of Attention**, but this time, the one-hot coding (to simulate embedding) is used to demonstrate how dot multiplication can refect the similarities between 2 vectors.



  - As can be seen here, using 1st row in A gives [.2, .9]; using 2nd row gives [.1, .4], and [.8, .3] for the third row, which serves as a table-lookup process. If considering the softmax algorithm, then the 1st element of the 1st row of matrix A has the highest possibility, so the queried value from matrix V (in this case, matrix B), is closest to the 1st row of matrix V, and the same for other queries.

- **First Order Sequence Model**
  - Given the following example:

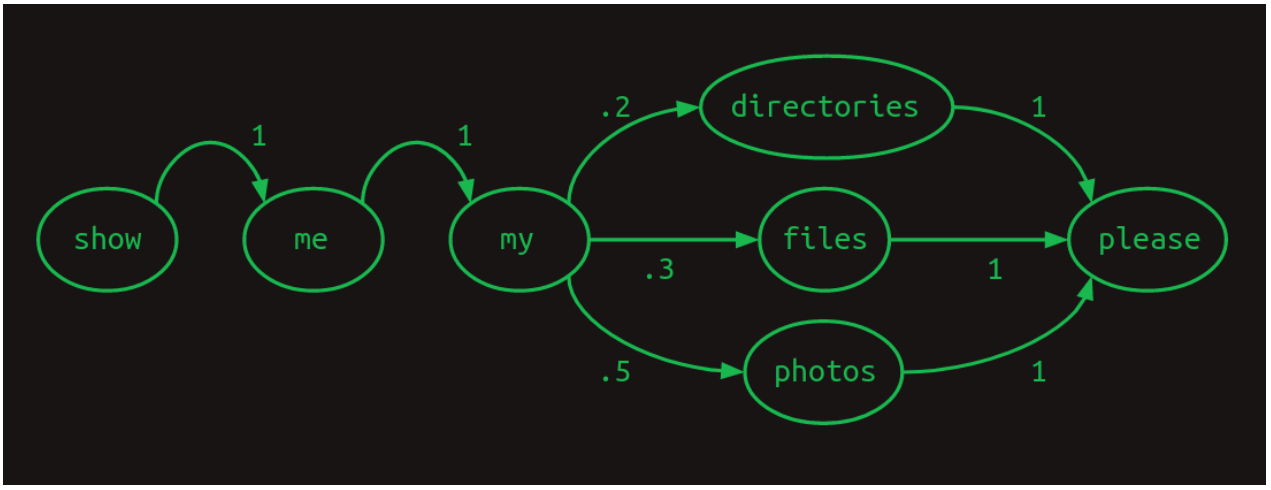
```
Show me my directories please.
Show me my files please.
Show me my photos please.

Our vocabulary size is now seven:
{directories, files, me, my, photos, please, show}.
```

Given a distribution:

```
directiories --- 20%
files        --- 30%
photos       --- 50%
Other parts have a probability of 100%.
```

Graphically,



This transition model is called a **Markov chain**, which contains the **Markov Property: the next state only depends on the current state.** Formally,

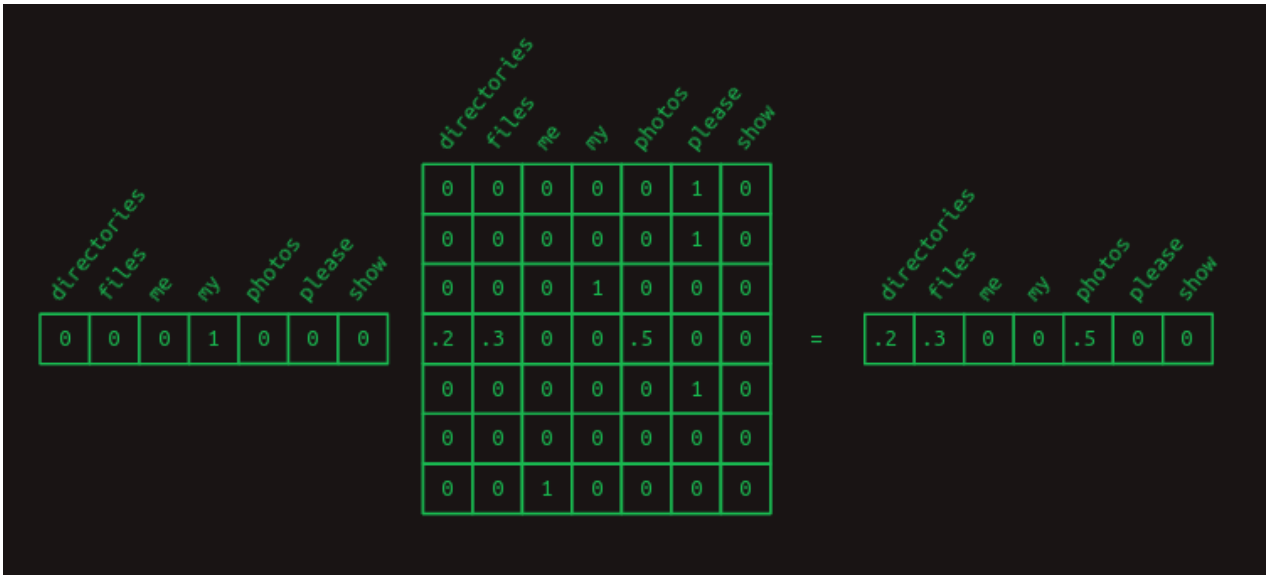
$$P(X_{t+n} = x | X_t, X_{t-1}, X_{t-2}, \ldots, X_{t-k}) = P(X_{t+n} = x | X_t)$$

- Markov Chain can be expressed by the matrix:

	directories	files	me	my	photos	please	show
directories	0	0	0	0	0	1	0
files	0	0	0	0	0	1	0
me	0	0	0	1	0	0	0
my	.2	.3	0	0	.5	0	0
photos	0	0	0	0	0	1	0
please	0	0	0	0	0	0	0
show	0	0	1	0	0	0	0

In the plot above, it can be seen that most of the numbers are 0 and 1, except "files", "directories", and "photos". The words near each row are the previous tokens (given), and the words near each col are the to-be-predicted tokens. In this case, in the row of "me", only "my" has a non-zero value (1), as the ONLY possible following word is "me" according to the example provided above. As for "my", there are 3 possible following tokens: "directories", "files", and "photos". The numbers in the corresponding cells are the possibility of each one of the three tokens.

- From what we have mentioned, we can regard each row as a query and use matrix multiplication to pull out the transition probabilities associated with any given word. For example:

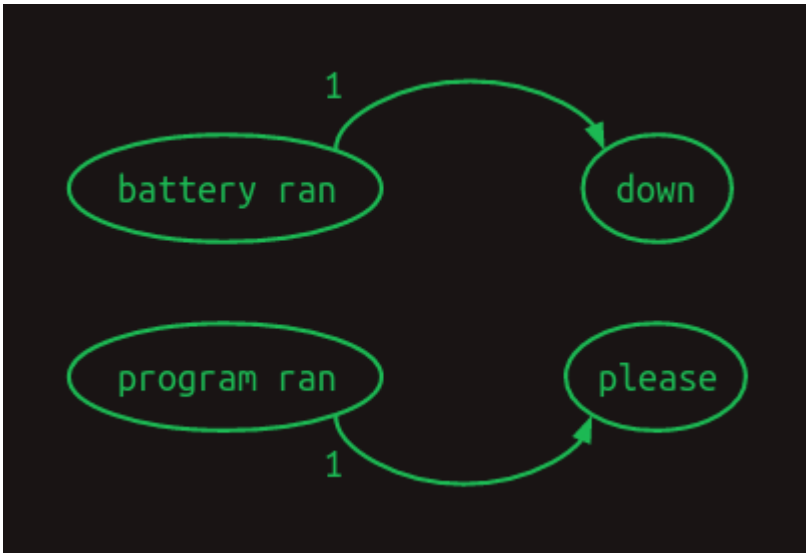


- **Second Order Sequence Model**
  - In some cases, the model can perform better when it can look back at 2 tokens. For instance:

```
Check whether the battery ran down please.
Check whether the program ran please.
```

Here, if the previous word is "ran", then it's hard to predict the next word. Where is this "ran" from? "battery"? "program"?

  - A good way to solve this is to look back at 2 words, which give:



  - Furthermore, in this case, the rows of the transition matrix have the following properties:
    1. Each row represents an **unordered bigram**.

$\text{len}(\text{tuple}) = 2 \text{ and } (a, b) \neq (b, a)$

    - **All** possible pairs are included.

So the resulting matrix will have $O(n^2)$ rows.

  - The result matrix will be:

	battery	check	down	please	program	ran	the	whether
battery ran	0	0	1	0	0	0	0	0
check whether	0	0	0	0	0	0	1	0
program ran	0	0	0	1	0	0	0	0
the battery	0	0	0	0	0	1	0	0
the program	0	0	0	0	0	1	0	0
ran down	0	0	0	1	0	0	0	0
whether the	.4	0	0	0	.6	0	0	0
⋮	0	0	0	0	0	0	0	0

(Note that the row labels in the figure above are not a good set of examples; it might seem to be confusing as it only contains consecutive pairs; non-consecutive pairs should also be included.)

- **Second order sequence model with skips**
  - What if the two sentences are extremely similar?

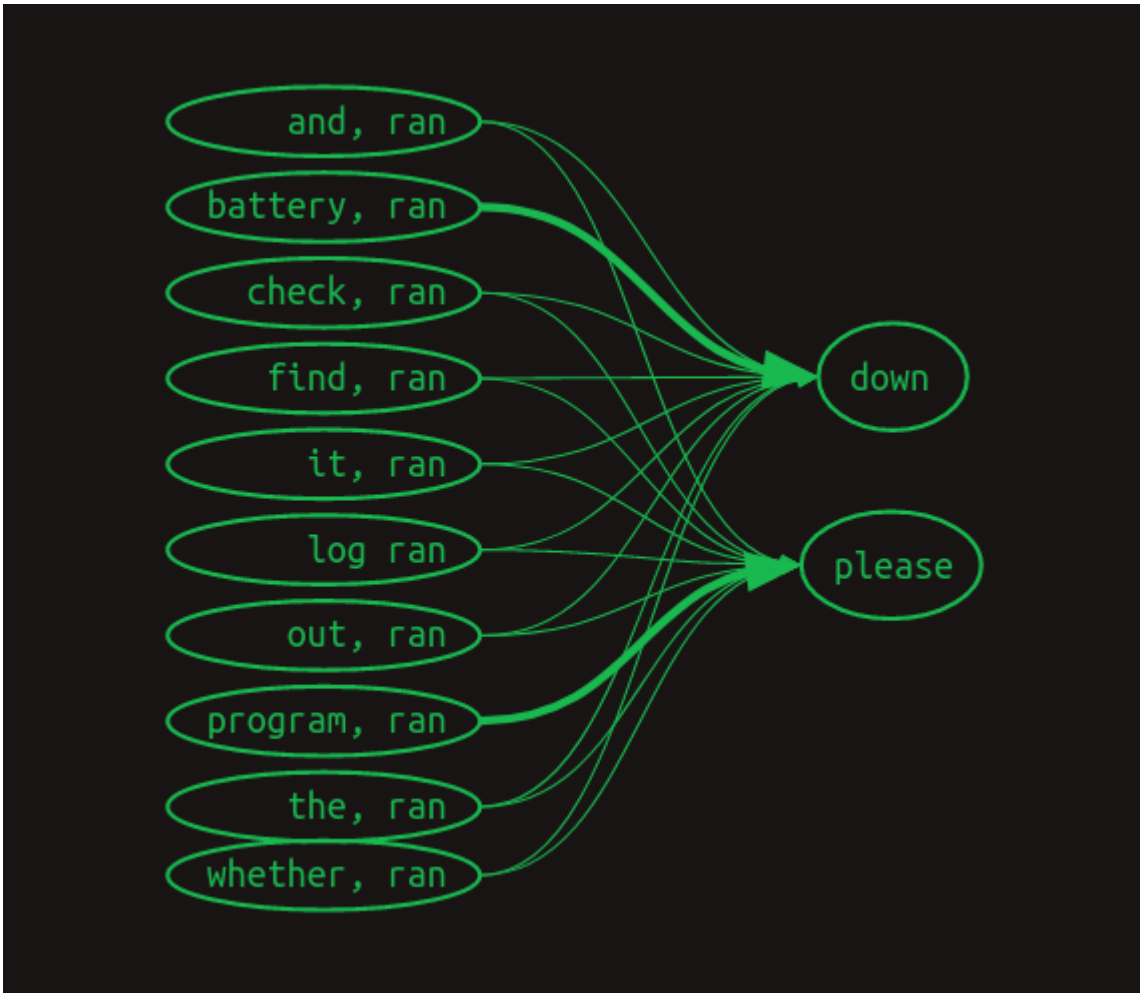
```
Check the program log and find out whether it ran please.
Check the battery log and find out whether it ran down please.
```

In this case, we must go back to check 8 words! Then, there are N^8 rows in the transition matrix! Horrible!

  - For a word W_n, we still use the 2nd order sequence model, but in this case, we only use the word before it to form the bigrams, which will capture **long range dependencies** and meanwhile discarding most useless pairs. Formally:

$$\{(W_1, W_n),\ (W_2, W_n), \ldots,\ (W_{n-1}, W_n)\}$$

  - From the example above, consider the word "ran", we have:



Draw the transition matrix, we have:

	and	battery	check	down	find	it	log	out	please	program	ran	the	whether
and, ran				.5					.5	0	0	0	0
battery, ran				1					0		0	0	0
check, ran				.5					.5		0		0
down, ran										0	0	0	0
find, ran				.5					.5	0	0	0	0
it, ran				.5					.5	0	0	0	0
log, ran				.5					.5		0	0	0
out, ran				.5					.5	0	0	0	0
please, ran										0	0	0	0
program, ran				0					1	0	0	0	0
ran, ran										0	0	0	0
the, ran				.5					.5		0	0	0
whether, ran				.5					.5	0	0	0	0

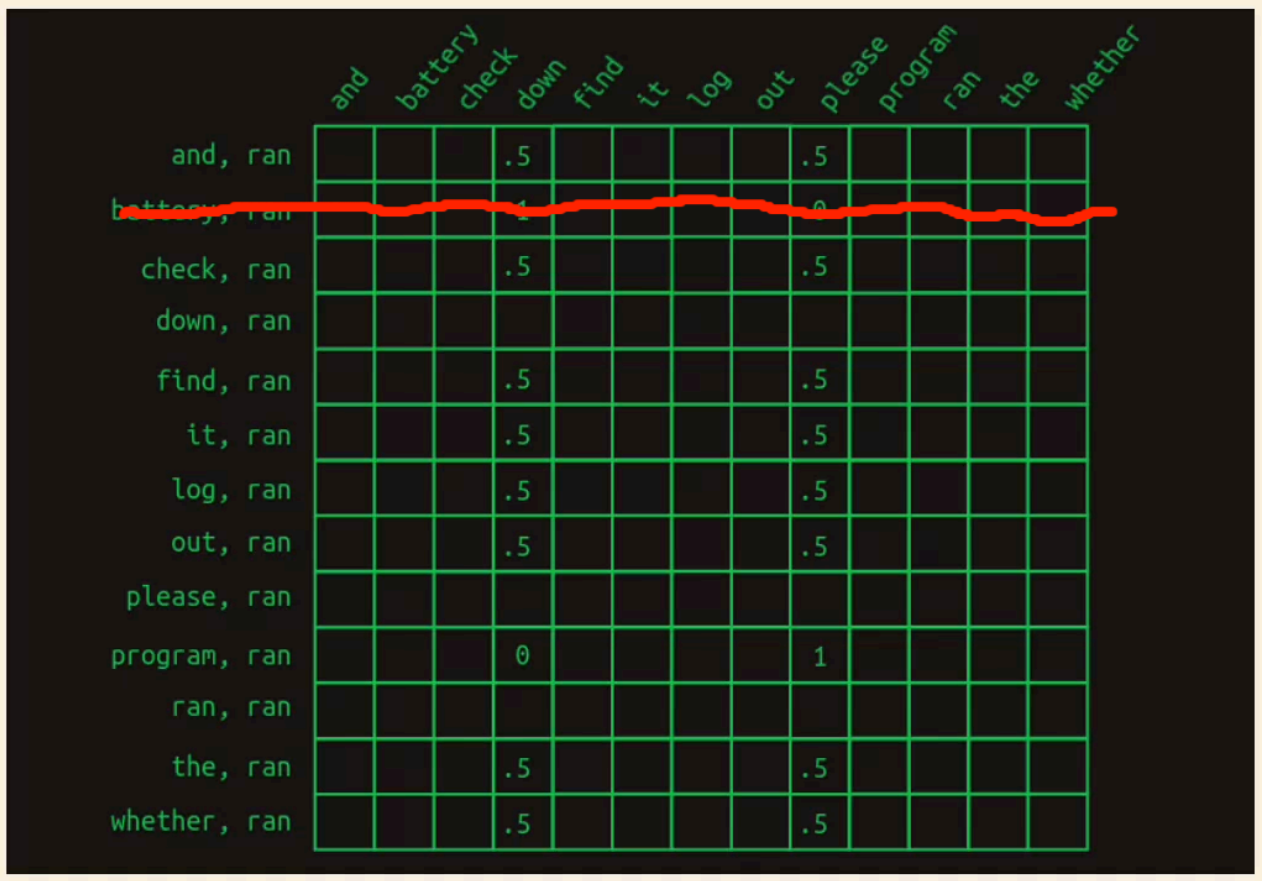
Note that this is NOT a markov chain as it DOESN'T ONLY depend on the current state. Instead, each

row represents one of many **features** that may describe the sequence at a particular point.

- Next, for **each possibility**:

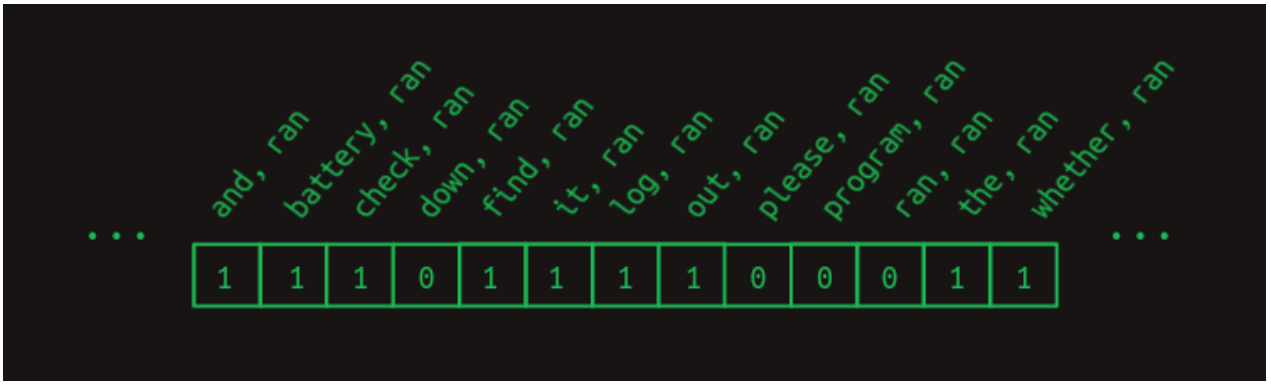
```
Check the program log and find out whether it ran
Check the battery log and find out whether it ran
```

We sum the number up to get a score for each. Remember, for the "battery" branch, we shouldn't add the "program, ran" in, and the similar for the "program" branch (the "program" branch is shown below).



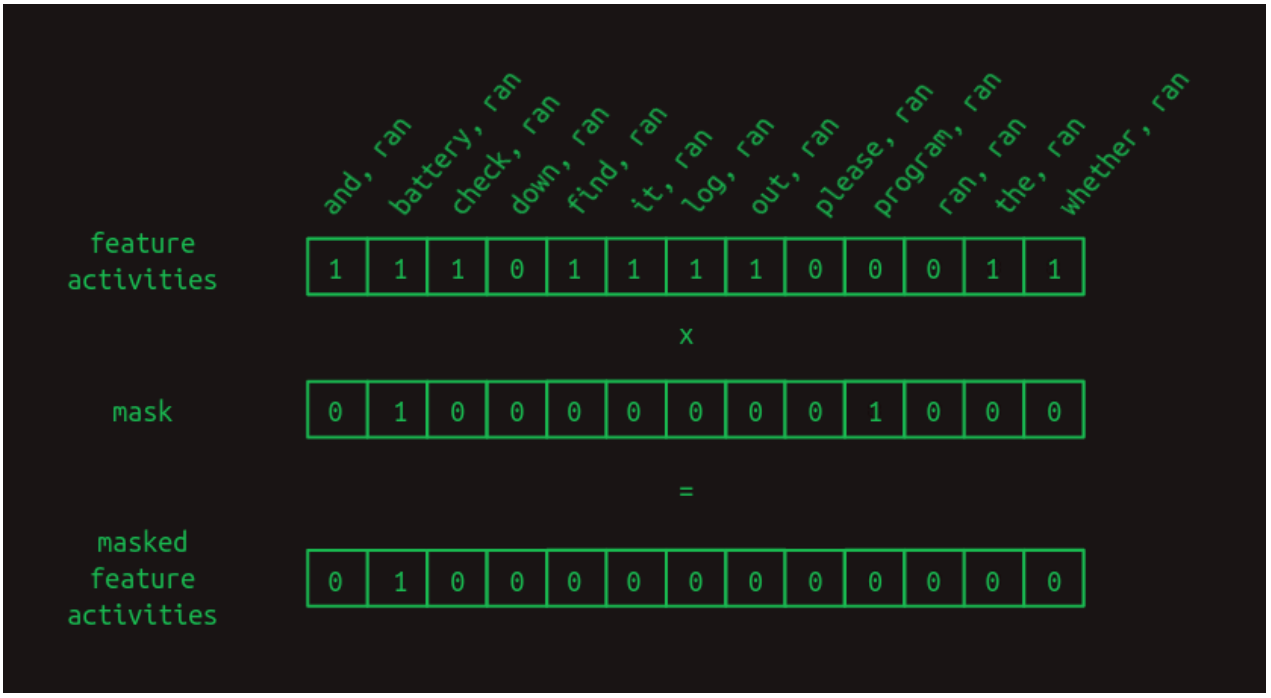
The score is $[\ldots, 4, \ldots, 5, \ldots]$ for the "program" branch and $[\ldots, 5, \ldots, 4, \ldots]$ branch. This can reflect that, according to the long range dependencies, the next word for "ran" of the "program" branch is "please", and of the "battery" branch is "down".

- **Masking**
  - The above process is actually matrix multiplication. Here, the feature vector being used to query the "bettery" branch is (notice that the "program, ran" is set to 0; all other "future" tokens are set to 0):



  - However, we are still keeping too much useless information here: the pairs other than (battery,

ran). These pairs all have the same possibilities (1). We can create another vector with the same dimension, but only with 1 kept on the features we CARE ABOUT (in this case, [battery, ran] and [program, run]; as we are considering the vector of "battery" branch, there will still be a 0 in the final masked vector for [program, run] pair). Then, we calculate the Hadamard product by multiplying the feature vector and the mask vector, resulting in:



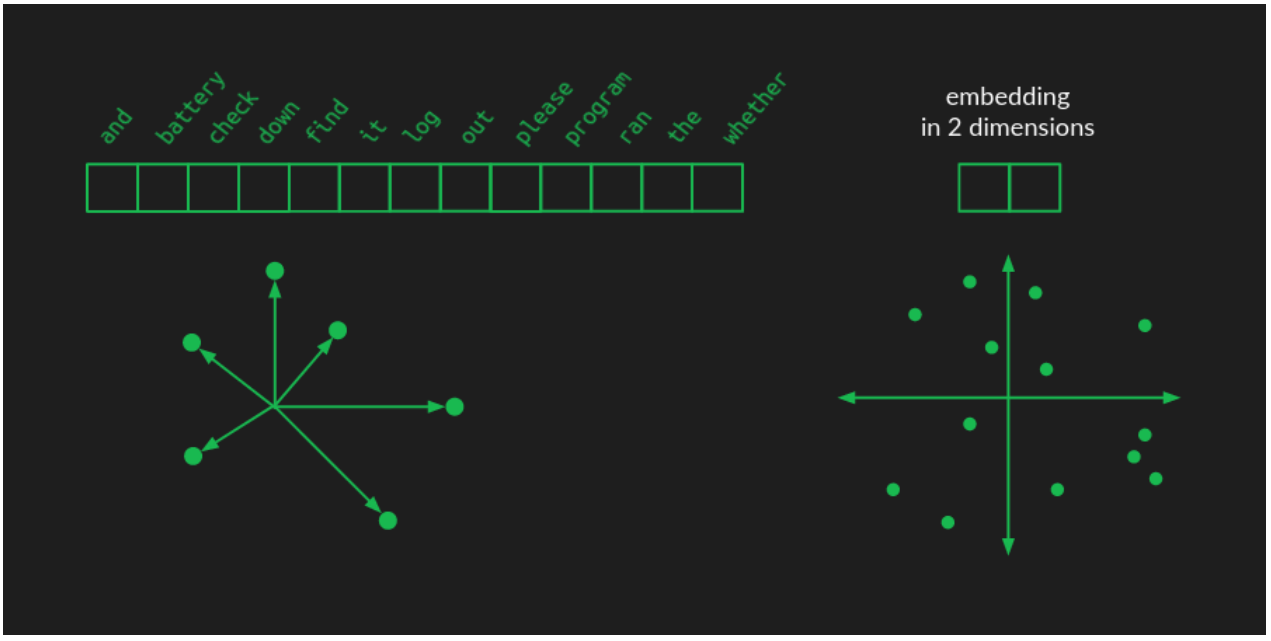
(Note that [program, run] is again 0)

- Using this masked-feature vector to do the same query we did above. For the "battery" branch, we will have [1, 0], and [0, 1] for the "program" branch.
- Compare with [5,4] and [4,5], the percentage of difference is amplified.
- **"This process of selective masking is the attention called out in the title of the original paper on transformers."**

- **Attention as matrix multiplication**
  - As computers are always good at matrix multiplication, we can stack the mask as a matrix, and use the interested feature vector to query the feature-specific mask (as shown in the image above).
- **Second order sequence model as matrix multiplications**
  - [...]
- **Sequence Completion**
  - To make decoders generate long sequences, what we also need is prompt. Prompt-crafting has its own art, which is called "prompt engineering".
  - After the prompts are passed into the model, the model will generate the next word by taking the highest possibility (greedy is one straight forward way) from the calculated distribution and then adding this word to the input.

- **Embeddings**
  - In a one-shot representation of a language, there is one vector element for each word. For a vocabulary of size *N*, that vector is an *N*-dimensional space.

- This will greatly increase the number of parameters needed, calculation complexity, and the cost of training. What we can do is **projecting** each one-hot representation into a lower-dimensional space, for example, 2.



- A good embedding model will be able to group words with similar meanings together. That means that whatever it learns to do with one word automatically gets applied to all the words right next to it, decreasing the training set size.
- Lower dimension means we need to discarding richness of the language, which is a trade-off to be considered.
- The process of converting one-hot encoding to embedding can also be realized by matrix multiplication.

- **Positional Encoding**
  - *(In this part, I personally don't like the description on the page; instead, I took advantage of ChatGPT's definition and provided some examples either from ChatGPT or from a book I read before.)*
  - Transformers process input sequences **without any inherent order** because they use a self-attention mechanism, but natural languages are usually strictly ordered.
  - So we need to use an encoding to record the position of the token.
  - There are 2 types of positional encoding:
    - Absolute: only depends on its absolute position in the sequence
      - Example: sinusoidal position encoding

      $PE(p, 2i) = \sin\left(\frac{p}{10000^{\frac{2i}{d}}}\right)$

      $PE(p, 2i+1) = \cos\left(\frac{p}{10000^{\frac{2i}{d}}}\right)$ where p is the position, i is the dimension, d is the total embedding dimension. In this case, for odd-numbered dimensions, we use cos, and sin for even-numbered dimensions to ensure uniqueness of position encoding.
    - Relative: relative to other tokens
      - Example: RoPE (5.2.4, page 90).

- **De-embeddings and softmax**
  - Opposite to embeddings, when everything is done, we must project (aka matrix multiplication)

the result matrix back to the initial one-hot like matrix.

- However, because of how projection to higher dimensional spaces works, the values associated with the other words won't be zero. The vector is still dense: some words may have similar value to the word with highest value, others may have smaller or negative values.
- One way to solve this problem is just to use **argmax**, the argument (element) that gives the maximum value.
- Another way is using softmax, as
  1. It produces probabilities.
  2. It's differentiable, which allows us to use it with backpropagation to train our transformer.
  3. When two words have significant differences, softmax will amplify the difference. However, if there are several words that all come out close to the top, it will preserve them all as highly probable, rather than artificially crushing close second place results.
- When several words have similar high probabilities, it's better to check all of these, as adding a touch of variety can result in something more interesting.

- **Multi-head Attention...Why do we want this?**
  - Normally, when we use softmax for attention, the model will tend to focus on ONLY a single item.
  - One way to solve this is to introduce multiple heads to calculate attention using different parameters. However, this will produce another problem: the increment of computational complexity.
  - We can use three matrices $W_Q, W_K, \text{and } W_V$ to project the embedded input to a lower-dimensioned $Q, K, \text{and } V$ matrices.
  - If we have h heads, then $d_K = d_Q = (normally)\ d_V = \frac{d_{model}}{h}$, though $d_Q = d_K \neq d_V$ sometimes.
  - Each head has its own (unique) $W_Q, W_K, \text{and } W_V$, so it can pay attention to DIFFERENT tokens. Sometimes we call space after the projection "sematic space".
  - Each head will have an output of $(n,\ d_V)$. We do $concat\ _{i=0}^{h}\ output_i$. Normally, this will give us a matrix with the same dimension as the embedded input, (n, d_{model}). In some special cases, we need to linear-transform it to fulfil this goal.

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O \\ \text{where head}_\text{i} &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned}$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

- **Let's look back to the single head...**
  - The result of $Q \cdot K^\top$ will be scaled by $\sqrt{d_K}$ to avoid it increasing crazily. Softmax will map this to a probability to perform as a filter: it will be applied to the value matrix V (where a token is "actually" represented) and shines a spotlight on the one prior element that is most useful to be aware of.
  - **The head calculates attention for every element of our input sequence, for every word in our sentence, not just the most recent word.** Though the future words aren't coming, and the past words have been determined, these attentions will still play a role in the next-word-

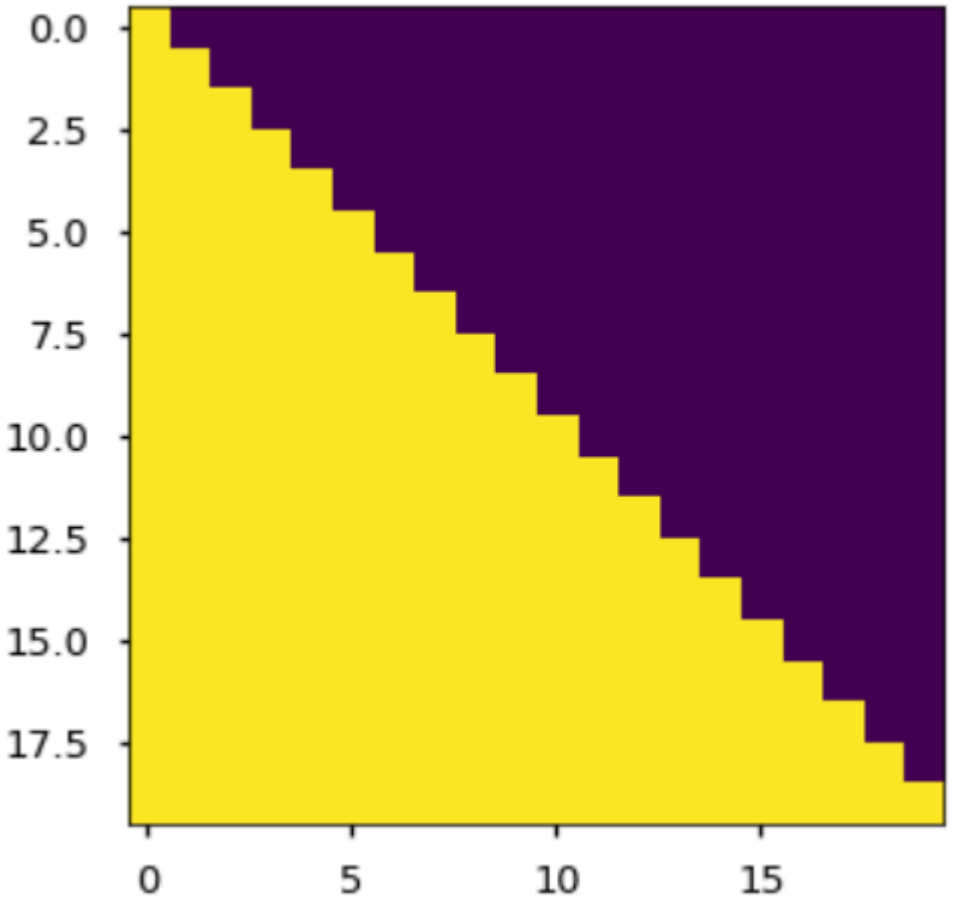
prediction, so we keep them. At the end of the day, we will discard most of them and highlight only ONE most important word.

- The causal mask, on the other hand, is a mask that makes the head "causal", that is, it cannot look to the future. In implementation, we manually set the attention paid to all words past the current position to negative infinity. "It avoids introducing any weird artifacts from imaginary future words."
    - For instance:

```
How are you? XXXXXXXXXX
How are you? I XXXXXXXX
How are you? I am XXXXX
How are you? I am fineX
How are you? I am fine.
```

In this case, "X"s are causal masks.

An image can also illustrate this:



Here the yellow part is the can-be-seen part, and the purple part is the masked part.

- **Skip connection**
    - Also known as residual connection
    - When we train a model, during backward-propagation, there exists a lot of "ridges" and "valleys" such that the gradient ~ 0 but is definitely not the best. We need to keep a "basic step" to make sure it can "step over" those local ridges and valleys.

- Residual connection is to add the original copy of values to the processed values, formally:

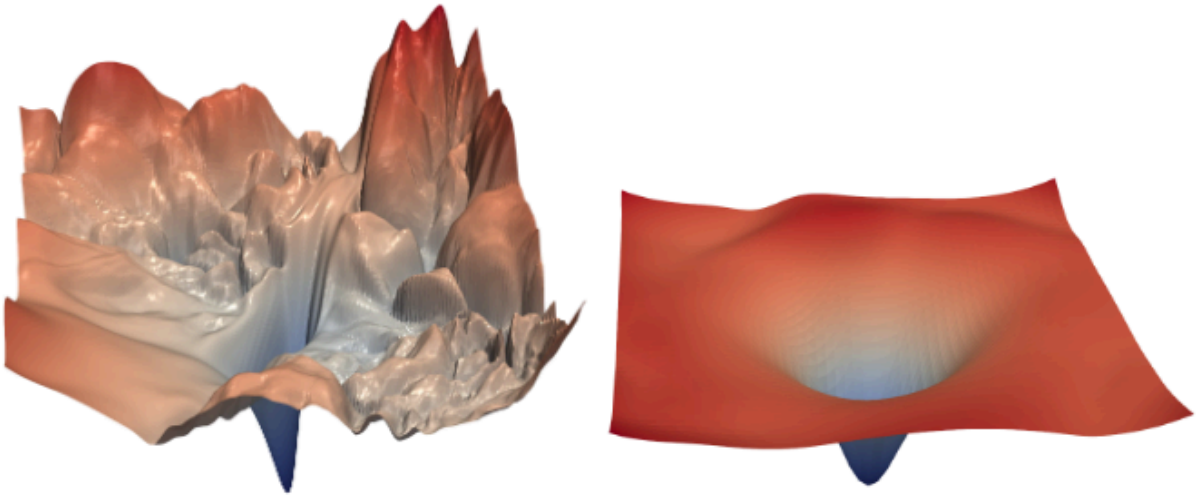
$$\mathbf{h}_{l+1} = \mathbf{h}_l + \mathcal{F}(\mathbf{h}_l; \theta_l)$$

- We write the backward propagation version of this function:

$$\frac{\partial L}{\partial \mathbf{h}_l} = \frac{\partial L}{\partial \mathbf{h}_{l+1}} \cdot \left(I + \frac{\partial \mathcal{F}(\mathbf{h}_l; \theta_l)}{\partial \mathbf{h}_l} \right)$$

Here we can see no matter how small $\frac{\partial \mathcal{F}(\mathbf{h}_l; \theta_l)}{\partial \mathbf{h}_l}$ is, it will keep a gradient of $\frac{\partial L}{\partial \mathbf{h}_{l+1}}$ to avoid being trapped into the local ridge/valleys.

- However, if it has reached the global optimum, then $\frac{\partial L}{\partial \mathbf{h}_{l+1}}$ ~ 0, then the gradient will also ~ 0.
- Graphically, we can see that with residual connection, the gradient graph is much smoother:



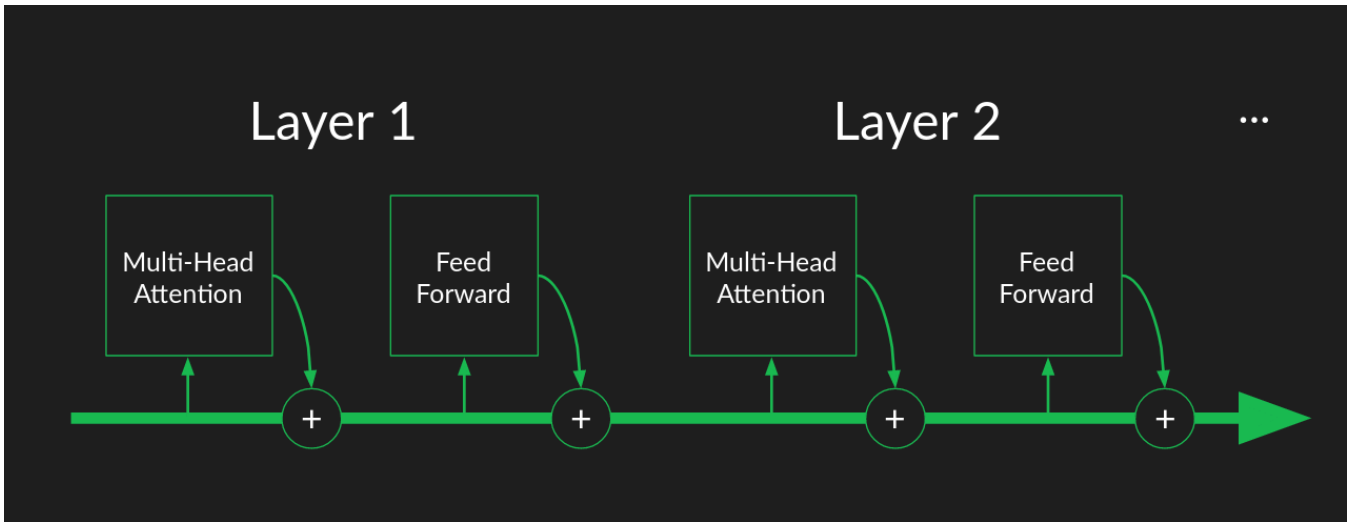
(a) without skip connections (b) with skip connections

- Along with skip connection, we use normalization most of the time. NNs, the basic components of LLMs, are inherently nonlinear, which "makes them very expressive but also sensitive to signals' magnitudes and distributions", so it's an art to balance and scale the values.
- Normalization is a technique that scales the values of the matrices to have a mean of 0 and st.d. of 1.
- This technique will accelerate the convergence of the model. With large values, the gradient can also be too large, and vice versa. In these two situations, gradient descent can be unstable, causing a slow convergence. One naive but extremely intuitive example is provided here.

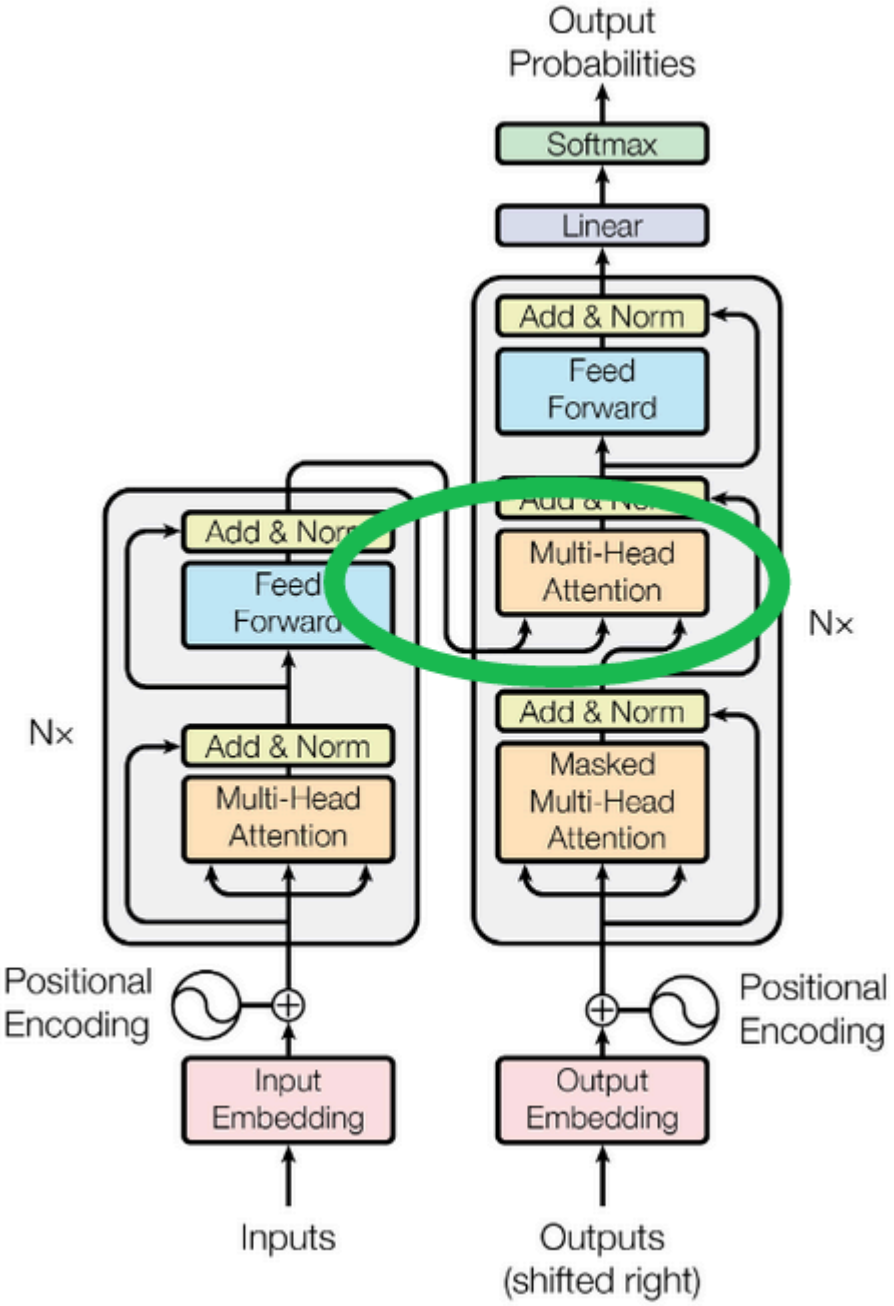
- **Multiple Layers**
  - Unlike the matrices we used above, normally, the randomly initialized matrices contain non-zero values; this means we have a long way to go to find the final, optimal values.
  - The convergence speed is slow when there are only a few routes to the optimum; if there are multiple, then the speed will increase.
  - With fewer attention layers, each value in the matrices must reach the optimal value. This is a long and mistake-proned process. However, if we have **multiple attention layers**, then this will no longer be a problem. If one layer produces any mistake or absurd result, the **skip connection** will be able to "eliminate" the effect of these layers, increase the robustness of the model.
    - *An intuitive way to understand this is: each layer is like a worker, and they need to manipulate the product. No matter whether they are expert or noobs, they will put the modified product into the pipeline and pass it to the next worker. If the last step is fine, then everything's good;*

otherwise, the next worker, probably an expert, will try to fix the problem. Then, the probability of having a bad final product will decrease.

- However, **it is not always beneficial to increase layers; after ~6 layers, the performance may decrease.**



- **Cross Attention**
  - Cross Attention needs the participation of two stacks: **the encoder stack and the decoder stack.** Unlike decoders, encoders will only output something quite abstract: a series of vectors in the embedding space. On a high level (but not precisely), these vectors contain important semantic information that can be further used, or, "extremely abstracted sentences".
  - Unlike decoders, **encoders have no causal masks**, meaning that they can "see" all of the words in the sentence.
  - Cross attention, just like normal self-attention, also contains $K, Q, \text{and } V$ matraces. However, ONLY the Q matrices are from the previously-generated token series; K, V matrices are projected from the output of the encoder by multiplying W_K, W_V matrices in the decoder.
  - This mechanism guarantees that the output of the encoder (input-wise global information) can be transferred to the decoder.
  - This process usually happens in seq2seq transformers, where both the encoder and the decoder are included. This type of model can handle translation and summerization tasks.



- **Embedding**
  - As we mentioned earlier, we can represent the vocabulary using one-hot vector; however, this is not practical: we need to know the size of our vocabulary, including each word's variance, such as past tense, plural, etc. This will give us a matrix with too many columns.
  - It is also not ideal to use character-level one-hot embedding, as most single characters have no semantic meanings on which the transformer works.
  - One commonly-used way to solve this problem is the **BPE embedding algorithm**.

```
Given a text corpus, and a set of initial tokens (normally single characters)
Pair and merge 2 tokens in the set such that the result has the HIGHEST frequency in the text corpus.
Add the merged tokens to the set.
Repeat until set.length reaches the given upper limit.
```

An example can be seen here, pages 72-73.

- Some advantages:
  - Agile. This algorithm makes the vocabulary set extendable. That is, when encountering a **new word**, BPE can process it **by breaking it down into its fundamental components (characters or subwords)** that are already in the vocabulary, which ensures that the model can **handle unseen words** without requiring an explicit new entry in the vocabulary.
  - Robustness. This algorithm is based on **vocabulary frequency statistics**, so it is not likely to add "fjhsosn" into the token unless it has a very high frequency in the given text corpus. This prevents unnecessary token expansion and keeps the vocabulary **efficient**.

- **Audio Input**
  - Unlike texts, where we need to convert them to dense float vectors (embedding), audio vectors are naturally dense vectors.
  - However, each audio vector is unique: it's nearly impossible to find similarities between two audio vectors. Also, vectors are too long. We need to find a way to compress the audio.
  - We can use Mel-frequency cepstrum，MFCC, to achieve our goal. Plus, MFCC can simulate the hearing perceptions of human beings, allowing the transformer to work in the perspective of humans.
  - Other signals, such as sensor signals, financial trends, etc, can also be used as train sets of transformers with simple transform, as they are all dense float vectors in nature.

- **Work Cited**

  - https://e2eml.school/transformers.html
  - https://kazemnejad.com/blog/transformer_architecture_positional_encoding/